

5 单件模式

独一无二的对象

告诉你，她是独一无二的，
看看这线，这弧度，这身体，
这车头灯！

你说的是我，还是车子？哦，
什么时候把烤箱手套还我？



我们的下一站是单件模式 (Singleton Pattern)，用来创建独一无二的对象，只能有一个实例的对象。你可能很高兴知道，单件模式的类图是所有模式中最简单的。事实上，图上只有一个类！但是不要掉以轻心，尽管从类设计的视角来说很简单，实现上还是需要一些深度的面向对象思想。因此，戴上思考帽，我们出发吧。



开发人员：这有什么用处？

大师：有很多我们只需要一个对象的情况：线程池、缓存、对话框、处理偏好设置和注册表的对象、日志对象、充当打印机、显卡等设备的驱动程序的对象。事实上，对于这种对象类型，如果我们实例化超过一个对象，就会碰到各种问题，像程序的行为异常、资源使用过量或者不一致的结果。

开发人员：OK，因此，也许有的类应该只被实例化一次，但我需要整章说这个吗？难道不能靠约定或全局变量做到？你知道，像在Java中，用静态变量可以做到。

大师：在许多方面，单件模式是一个约定，用来确保为给定类实例化独一无二对象。如果你有更好的做法，对大家是个好消息，但是，记住，像所有模式一样，对于确保只有一个对象被创建，单件模式是一个经过时间考验的方法。单件模式也给我们一个全局的访问点，就像全局变量一样，但没有全局变量的缺点。

开发人员：什么缺点？

大师：嗯，举个例子：如果你把一个对象赋值给一个全局变量，那么这个对象可能在应用启动时就被创建，对吧？要是这个对象是资源密集型的，而你的应用一直没用到它，那怎么办？稍后会看到，有了单件模式，我们可以在需要时才创建对象。

开发人员：这看起来也应该没那么难。

大师：如果你熟练掌握静态类变量、方法以及访问修饰符，那就不难。但是，不管使用哪一种方式，了解单件的运作方式还是有意思的，而且，单件模式尽管听起来简单，正确编写单件代码却很难。只需要问一下你自己：怎样防止多于一个对象被实例化？答案不是那么显而易见的，对吧？

小小单件

“The Little Lisper” 风格的苏格拉底式练习

怎样创建单个对象？

`new MyObject();`

那，要是另一个对象也要创建一个MyObject怎么办？
可以再次在MyObject上调用new吗？

当然可以。

因此，只要有某个类，我们是否总是能一遍遍地
实例化它？

是的。哦，仅对公开的类。

如果不是公开的类呢？

嗯，如果不是公开的类，只有同一个包内的类
可以实例化它，但还是可以实例化多次。

嗯，有意思。

不知道。我没想过，但我猜想这行得通，因为
这是合法的定义。

你知道可以这样做吗？

```
public MyClass {  
  
    private MyClass() {}  
  
}
```

什么意思？

我认为它是一个不能实例化的类，因为它有
一个私有的构造器。

嗯，有没有对象可以使用私有的构造器
呢？

嗯，我想MyClass内的代码是唯一能调用此构
造器的代码，但是这又不太合乎常理。

为什么不合常理？

因为我必须得有一个该类的实例来调用它，但我没法有实例，因为没有其他类可以实例化它。这是一个“鸡生蛋，蛋生鸡”的问题：我可以使用来自类型MyClass的对象的构造器，但我没法实例化这个对象，因为没有其他对象可以使用“new MyClass()”。

OK。只是想想而已。

MyClass是一个有静态方法的类。我们可以像这样调用静态方法：

```
MyClass.getInstance();
```

这个怎样？

```
public MyClass {  
  
    public static MyClass getInstance() {  
    }  
}
```

为什么使用MyClass，而不是某些对象名称？

嗯，getInstance()是一个静态方法。换句话说，它是一个类方法。你需要使用类名称来引用静态方法。

非常有意思。要是把这些凑在一起会怎样？

哇哦，你肯定可以。

现在我可以实例化一个MyClass了吗？

```
public MyClass {  
  
    private MyClass() {}  
  
    public static MyClass getInstance() {  
        return new MyClass();  
    }  
}
```

因此，现在你能想到第二种实例化对象的方式吗？

```
MyClass.getInstance();
```

你可以完成只有一个MyClass实例被创建的代码了吧？

是的，我认为应该可以……

（代码在下页。）

剖析经典的单件模式实现



注意!

如果你只是跳着翻到这一页，不要盲目地键入代码。本章稍后会指出这份代码的一些问题。

把类名从MyClass改为Singleton。

```
public class Singleton {
    private static Singleton uniqueInstance;

    // 其他有用的实例变量写在这里

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }

    // 其他有用的方法写在这里
}
```

一个静态变量持有Singleton类的唯一实例。

构造器声明为私有，只有Singleton可以实例化这个类!

getInstance()方法提供了一种实例化该类的方式，也返回它的一个实例。

当然，Singleton是一个正常的类。它有一些其他用途的实例变量和方法。



Code up Close

uniqueInstance持有唯一实例，记住，它是一个静态变量。

如果uniqueInstance为空，表示还没有创建实例……

……如果不存在，我们通过其私有的构造器实例化Singleton，并把它赋值给uniqueInstance。注意，如果我们不需要这个实例，它就不会被创建，这就是延迟实例化（lazy instantiation）。

```
if (uniqueInstance == null) {
    uniqueInstance = new Singleton();
}
return uniqueInstance;
```

如果uniqueInstance不为空，说明之前已经创建过对象，直接跳到return语句。

当执行到这句代码，就有了一个实例，并返回。



模式曝光

本周的访谈：
单件的告白

HeadFirst: 今天我们很高兴访谈单件对象。一开始，不妨介绍一下你自己？

Singleton: 好吧，我很独特，只有一个我！

HeadFirst: 只有一个？

Singleton: 是的，一个。基于单件模式，这个模式确保在任何时候只有一个我的实例。

HeadFirst: 这不是一种浪费吗？有人花了时间来开发出一个成熟的类，现在，这个类只能有一个对象？

Singleton: 根本不会！“一个”的威力很强的。比如说，你有一个包含注册表设置的对象。你不想让这个对象有多个副本，到处赋值—这会导致混乱。通过使用像我这样的对象，你可以确保应用中的每个对象使用同一全局资源。

HeadFirst: 请继续……

Singleton: 哦，我擅长各种事情。你知道，有时候独身也有好处。我常常被用来管理资源池，像连接或者线程池。

HeadFirst: 我还是觉得，同类中只有你一个，好像有一点孤单。

Singleton: 因为只有我一个，所以一直很忙，但还是希望更多开发人员知道我。许多开发人员碰到bug，是因为有多个对象副本，他们却浑然不觉。

HeadFirst: 因此，允许我这么问，你怎么知道只有一个你？会不会什么人用new操作符创建一个“新的你”？

Singleton: 不会！我真的是独一无二的。

HeadFirst: 好吧，开发人员要不要发誓不会实例化多于一个的你？

Singleton: 当然不。真相是……好吧，但这牵扯到个人隐私……我没有公开的构造器。

HeadFirst: （大声）没有公开的构造器！噢，不好意思，没有公开的构造器？

Singleton: 没错。我的构造器声明为私有。

HeadFirst: 这怎么行得通？你怎样被实例化？

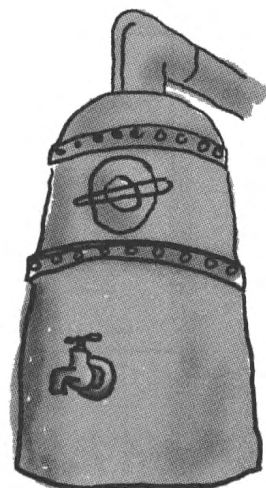
Singleton: 你了解的，为了获得一个单件对象，你不是实例化一个，只是请求一个实例。因此我的类有一个静态方法，称为getInstance()。调用这个方法，我就立刻现身，准备工作。事实上，当你请求我时，我可能已经在帮助其他对象了。

HeadFirst: 单件先生，看起来你的外表下面还有很多东西。谢谢你如此坦白，希望能很快地再和你交流！

巧克力工厂

大家都知道，现代化的巧克力工厂都有计算机控制的巧克力锅炉。锅炉把巧克力和牛奶融在一起，然后送到制造巧克力棒的下一个阶段。

这里有一个Choc-O-Holic公司的工业强度巧克力锅炉控制器类。查看代码，你会注意到代码写得非常小心，以防止不好的事情发生，像排出500加仑的未煮沸混合物，或者锅炉已经满了还继续放原料，或者烧一个空锅炉。



```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;
```

```
    public ChocolateBoiler() {
        empty = true;
        boiled = false;
    }
```

这段代码只有在锅炉空的时候才启动！

```
    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // 往锅炉里填充牛奶/巧克力混合物
        }
    }
```

要往锅炉里填充原料，锅炉必须是空的。一旦锅炉满了，我们就设置empty和boiled标志。

```
    public void drain() {
        if (!isEmpty() && isBoiled()) {
            // 排出煮沸的牛奶和巧克力
            empty = true;
        }
    }
```

要排空锅炉，锅炉必须是满的（非空）且煮过的。一旦排出完毕，我们把empty设置@true。

```
    public void boil() {
        if (!isEmpty() && !isBoiled()) {
            // 将炉内物煮沸
            boiled = true;
        }
    }
```

要煮混合物，锅炉必须是满而且未煮过的。一旦煮沸，我们把boiled标志设为true。

```
    public boolean isEmpty() {
        return empty;
    }
```

```
    public boolean isBoiled() {
        return boiled;
    }
```

```
}
```



Choc-O-Holic公司干了件体面的事，防止不好的事情发生，你不这么认为吗？再一次，你可能会质疑，要是逃逸出两个ChocolateBoiler（巧克力锅炉）实例，就会发生一些很糟糕的事情。

如果在一个应用中，多于一个ChocolateBoiler实例被创建，可能发生哪些很糟糕的事？



削尖你的铅笔

请帮Choc-O-Holic改进ChocolateBoiler类，把它转成单件。

```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;

     ChocolateBoiler() {
        empty = true;
        boiled = false;
    }

    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // 往锅炉里填充牛奶/巧克力混合物。
        }
    }

    // 剩下的ChocolateBoiler代码……
}
```


定义单件模式

现在你的大脑里已经有了单件的经典实现，是时候坐下来享受一条巧克力棒，细细品味单件模式了。

我们从该模式的简洁定义开始：

单件模式确保一个类只有一个实例，并提供一个全局访问点。

没有太让人吃惊的东西，但是让我们更深入地剖析：

- 到底怎么回事？我们让一个类管理自己的单个实例，同时也避免其他类自行创建新实例。想取得实例，只能通过该类。
- 我们也提供该实例的全局访问点：无论何时你需要一个实例，只要查询该类，它会交回单个实例。正如前面讲过的，我们可以实现这个，以便单件以延迟方式被创建，这对资源密集的对象特别重要。

OK，我们来看看类图：

getInstance()方法是静态的，这意味着它是一个类方法，因此你可以在代码的任何地方，方便地用Singleton.getInstance()访问它，就和访问全局变量一样容易，而且单件还有延迟实例化等优点。



uniqueInstance类变量持有唯一的单件实例。

实现单件模式的类不只是一个单件，它也是一般的类，有自己的一套数据和方法。

好时 ~~休斯顿~~，我们有麻烦了……

看起来巧克力锅炉让我们失望了。尽管我们用经典的单件来改进代码，但是不知怎么回事，在有一批牛奶和巧克力已经在加热的情况下，ChocolateBoiler的fill()方法竟然能够继续填充原料！这可是会溢出五百加仑的牛奶（和巧克力）！怎么回事？



添加线程会造成这样吗？不是只要设置uniqueInstance变量为ChocolateBolier的单个实例，所有到getInstance()的调用就应该返回相同的实例吗？对吧？

化身为JVM

我们有两个线程都要执行这段代码。你的工作是扮演JVM，判断是否可能有这种情况：两个线程持有不同的锅炉对象。

提示：其实你只需要看看`getInstance()`方法中的操作顺序以及`uniqueInstance`的值，看它们会怎么重选。使用代码磁贴帮助你研究代码怎么可能交错导致创建两个锅炉对象。



```
ChocolateBoiler boiler =
    ChocolateBoiler.getInstance();
boiler.fill();
boiler.boil();
boiler.drain();
```

```
public static ChocolateBoiler
    getInstance() {
```

```
    if (uniqueInstance == null) {
```

```
        uniqueInstance =
            new ChocolateBoiler();
```

```
    }
```

```
    return uniqueInstance;
```

```
}
```

确认你在翻页前，先看看188页的答案！

线程一

线程二

`uniqueInstance`
的值

处理多线程

通过把getInstance()变成同步（synchronized）方法，我们的多线程困境几乎可以轻易解决：

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // 其他有用的实例变量在这里  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
  
    // 其他有用的方法在这里  
}
```

通过增加synchronized关键字到getInstance()，我们迫使每个线程进入此方法之前，要先等待轮到它。也就是说，没有两个线程可以同时进入该方法。

我同意这可以解决问题。
但是同步导致性能开销大，
这也是一个问题吧？

说得对，实际上比你所想象的还糟糕一些：只有第一次执行此方法时才和同步扯上关系。换句话说，一旦设置uniqueInstance变量为单件的实例，就不再需要同步这个方法了。第一次之后，同步完全是累赘！



我们能改进多线程吗？

对于大多数Java应用，我们显然需要确保单件在多个线程的存在下正常工作。但是，同步getInstance()方法性能开销大，该怎么做？

好吧，我们有一些选项……

1. 如果getInstance()的性能对应用来说不是很关键，什么也不做。

没错，如果调用getInstance()方法没有给你的应用增加很重的负担，忘了它。同步getInstance()直白又高效。只要牢记，同步一个方法可能会使性能下降100倍，因此，如果你的代码中高频使用的部分开始用getInstance()，你可能不得不重新考虑了。

2. 转为急切 (eagerly) 创建实例，而不用延迟创建。

如果你的应用总是创建并使用单件实例，或者单件的创建以及运行时方面负担不重，你可以早点创建单件，像这样：

```
public class Singleton {
    private static Singleton uniqueInstance = new Singleton();

    private Singleton() {}

    public static Singleton getInstance() {
        return uniqueInstance;
    }
}
```

在静态初始化器中创建单件实例。这段代码保证是线程安全的！

我们已经有了一个实例，因此返回它即可。

用这个方法，当类被加载时，我们依靠JVM来创建单件的唯一实例。JVM 保证在任何线程访问静态uniqueInstance变量之前，实例会被创建。

3. 用“双重检查加锁”在getInstance()减少使用同步。

有了双重检查加锁，我们首先检查实例是否已经创建，如果没有，“才”同步。这样一来，只有第一次会同步，这正是我们要的。

我们来看看代码：

```
public class Singleton {
    private volatile* static Singleton uniqueInstance;

    private Singleton() {}
```

```
public static Singleton getInstance() {
```

```
    if (uniqueInstance == null) {
        synchronized (Singleton.class) {
            if (uniqueInstance == null) {
                uniqueInstance = new Singleton();
            }
        }
    }
}
```

```
    return uniqueInstance;
```

```
}
```

```
}
```

检查实例，如果没有，
进入同步区块。

注意，只有第一次才同步！

进入区块后，再检查一次。如果
依然是空的，创建一个实例。

* volatile关键词确保：当uniqueInstance变量
被初始化为单件实例时，多个线程正确处理
uniqueInstance变量。

如果在使用getInstance()方法时，性能是个问题，那么这个实现单件的方法可以大大减少负载。



注意！

双重检查加锁不适用1.4以及更早版本的Java!

如果出于某些原因，你在使用旧版的Java，不幸的是，在Java 1.4版以及更早版本中，允许不恰当同步。如果你必须使用比Java 5更早的JVM，就考虑单件的其他实现方法。

与此同时，回到巧克力工厂……

我们诊断完多线程问题，巧克力锅炉也被清理干净，准备开工了。但是，首先得修正多线程问题。我们手边有一些方案，每个方案都有优缺点，打算采用哪一个？



削尖你的铅笔

对于每个方案，描述其对修正巧克力锅炉代码问题的适用性：

同步getInstance()方法：

使用急切实例化：

双重检查加锁：

恭喜！

此刻，巧克力工厂的问题有了开心的结局，Choc-O-Holic很高兴有一些专业知识应用到锅炉的代码中。不管你应用哪个多线程方案，锅炉应该会处在良好的状态，不会再出事故了。恭喜，在本章中，你不但想办法逃离了500磅热巧克力的危机，也遍历了单件模式的所有潜在问题。

没有蠢问题

问：对这样一个只包含一个类的简单模式，单件看起来还是有一些问题。

答：嗯，我们提前警告你了！但不要让问题把你吓住了。虽然正确地实现单件比较棘手，但阅读本章之后，你会了解创建单件的技巧，在你需要控制所创建实例个数的地方，还是应该用上它们。

问：难道我不能创建一个类，把所有的方法和变量都定义为静态的？这不是跟单件一样吗？

答：是的，如果你的类是自包含的，不依赖于复杂的初始化，可以这样。但是，因为静态初始化的方式由Java把持，这可能会很混乱，特别是涉及多个类时。这么做常常会导致微妙的、难以发现的bug，这些bug和初始化的次序有关。除非有非常强烈的需要以这样的方式实现“单件”，否则还是停留在对象的世界里好得多。

问：类加载器怎么样？我听说两个类加载器，可能都有机会各自创建自己的单件实例。

答：是的，每个类加载器都定义了一个命名空间。如果有两个或多个类加载器，你可以多次加载同一个类（每个类加载器一次）。现在，如果这个类碰巧是一个单件，那么，因为我们有多于一个版本的类，我们就有多于一个的单件实例。因此，如果你在使用多个类加载器和单件时要小心。绕过这个问题有一个办法，自行指定类加载器。

问：反射和序列化/反序列化呢？

答：是的，反射和序列化/反序列化也可以出现单件的问题。如果你是一名使用反射、序列化和反序列化的进阶Java用户，你需要记住这一点。

问：之前我们谈到松耦合原则。单件不是违反了吗？毕竟，我们代码中，每个依赖于单件的对象会紧耦合到非常特定的对象。

答：是的，事实上，这是对单件模式常见的批评。松耦合原则说“尽力达到交互对象之间的松耦合设计”。单件很容易违反这个原则：如果你修改单件，你有可能不得不修改每个连接到它的对象。

问：我受到的教育一直是：类应该做一件事，而且只做一件事。类如果做两件事，就被认为是坏的设计。单件是不是也违反了这个？

答：你指的是单一责任原则，是的，你是对的：不只负责管理自己的唯一实例（并提供全局访问），而且主要还是在应用中担当角色。因此，你认为它承担了两个责任是没错的。尽管如此，由类管理自己的实例，这样的做法并不罕见，这样做使得整体设计更简单。另外，因为单件模式广泛使用，许多开发人员熟悉它。也就是说，一些开发人员确实感觉需要抽象出单件功能。

问：我要子类化我的单件代码，但我碰到问题。子类化一个单件合适吗？

答：子类化单件的一个问题是构造器是私有的。你不能扩展一个带有私有构造器的类。因此，你不得不做的第一件事是把构造器改成公开的或保护的，但是这么一来，它就不再是真正的单件了，因为其他类可以实例化它。

如果你确实修改了构造器，就有另一个问题。单件的实现基于静态变量，因此，如果你直接搞一个子类，所有派生类会共享同一个实例变量。这可能不是你想要的。因此，为了让子类化顺利运作，基类中要实现类似注册表的东西。

但你真正能从子类化一个单件得到什么呢？像大多数模式一样，单件不一定是适合进入一个库的方案。另外，给任何已有的类添加单件代码是很琐碎的工作。最后，如果你在应用中使用大量单件，你应该仔细看看你的设计。单件适用的场合并不多。

问：我还是没有完全理解，为什么全局变量比单件差。

答：在Java中，全局变量基本上是对对象的静态引用。以这种方式使用全局变量有一些缺点。我们已经提到了一个：延迟实例化 vs. 急切实例化。但我们需要牢记单件模式的意图：确保类只有一个实例存在，并提供全局访问。一个全局变量可以提供后者，但提供不了前者。全局变量也倾向于怂恿开发人员用大量对小对象的全局引用来污染命名空间。单件不鼓励这样做，但还是可能被滥用。



我刚认识到……通过使用枚举，我们可以解决许多单件问题。对吧？

哈，好主意！

我们讨论的许多问题（操心同步、类加载问题、反射和序列化/反序列化问题），都可以通过使用枚举来创建单件解决。做法如下：

```
public enum Singleton {
    UNIQUE_INSTANCE;
    // 更多有用的字段在这里
}

public class SingletonClient {
    public static void main(String[] args) {
        Singleton singleton = Singleton.UNIQUE_INSTANCE;
        // 在这里使用单件
    }
}
```

回到旧时光，当我们不得不来回走路上学、爬山、踏雪时，Java没有枚举。

耶，没什么可说的。这是目前为止最简单的单件了，对吧？现在，你可能会问，为什么我们之前要说一遍创建一个带getInstance()方法的单件类，然后同步之类的？目的是让你真真正正理解单件如何工作。既然你已经知道，你可以放下这些，在需要单件的任何时候使用枚举，而且还可以在Java访谈中出彩，如果被问到这样的问题：

“不用枚举，怎样实现单件？”



你能用枚举改写Choc-O-Holic吗？试一下。



你的设计工具箱中的工具

你添加了另一个模式到工具箱。单件给了你创建对象的另一种方法——创建唯一的对象

OO原则

封装变化。

组合优于继承。

针对接口编程，而不是实现。

努力达到交互对象之间的松耦合设计。

类应该对扩展开放，对修改关闭。

依赖于抽象，不依赖于具体类。

OO基础

对象
封装
静态
继承

OO模式

单件，确保一个类只有一个实例，并提供全局访问点。

当你需要确保应用中的某个类只有一个实例，就用单件模式吧。

要点

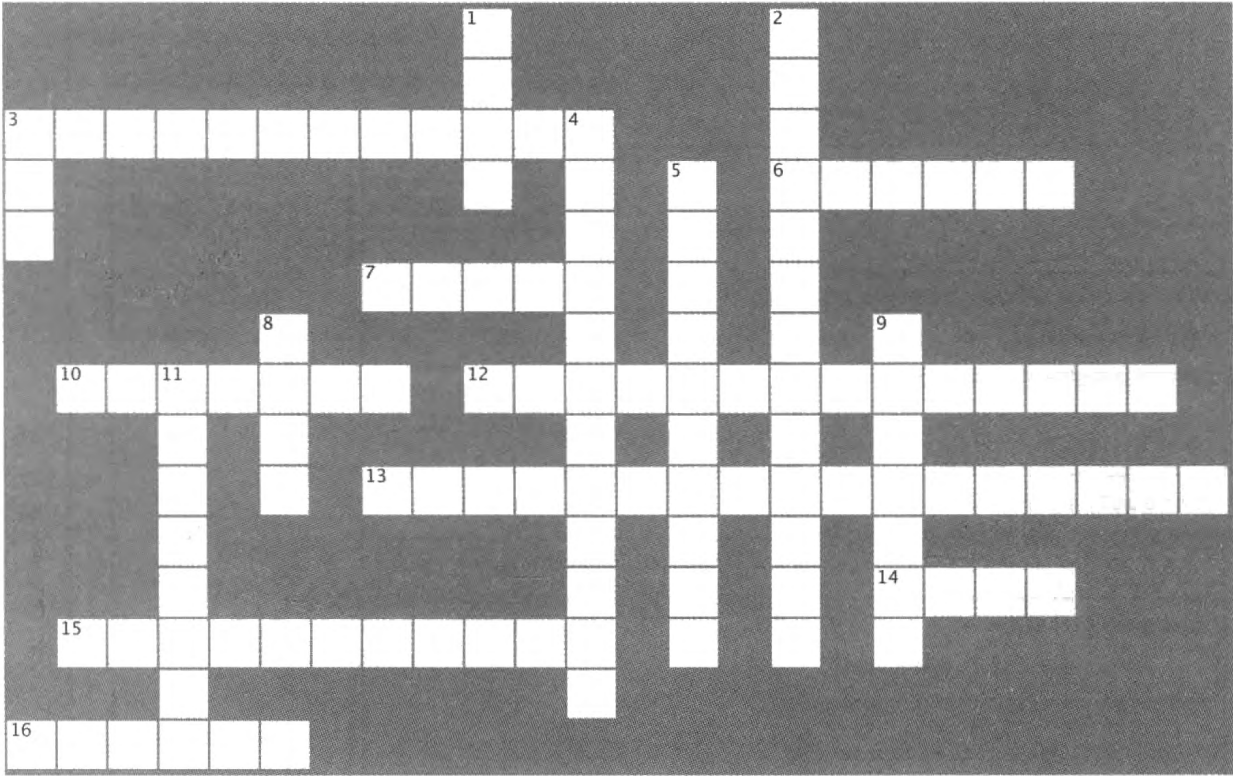
- 单件模式确保应用中一个类最多只有一个实例。
- 单件模式也提供访问此实例的全局点。
- Java的单件实现用了一个私有构造器、一个静态方法以及一个静态变量。
- 检查你的性能和资源约束，为多线程应用小心选择一个适当的单件实现（我们应该把所有应用都考虑为是多线程的）。
- 提防双重检查加锁实现。Java 5之前的版本，不是线程安全的。
- 如果你使用多个类加载器，要小心，可能导致单件实现失效，导致出现多个实例。
- 你可以使用Java的枚举来简化单件的实现。

正如你所看到的，尽管看起来简单，但单件实现中涉及到很多细节，不过，读完本章后，你可以准备出去到野外使用单件了。



设计模式填字游戏

坐下来，打开因为解决多线程问题而获赠的巧克力，花一点时间解决这个小小填字游戏，所有的答案词汇都来自本章。



横排

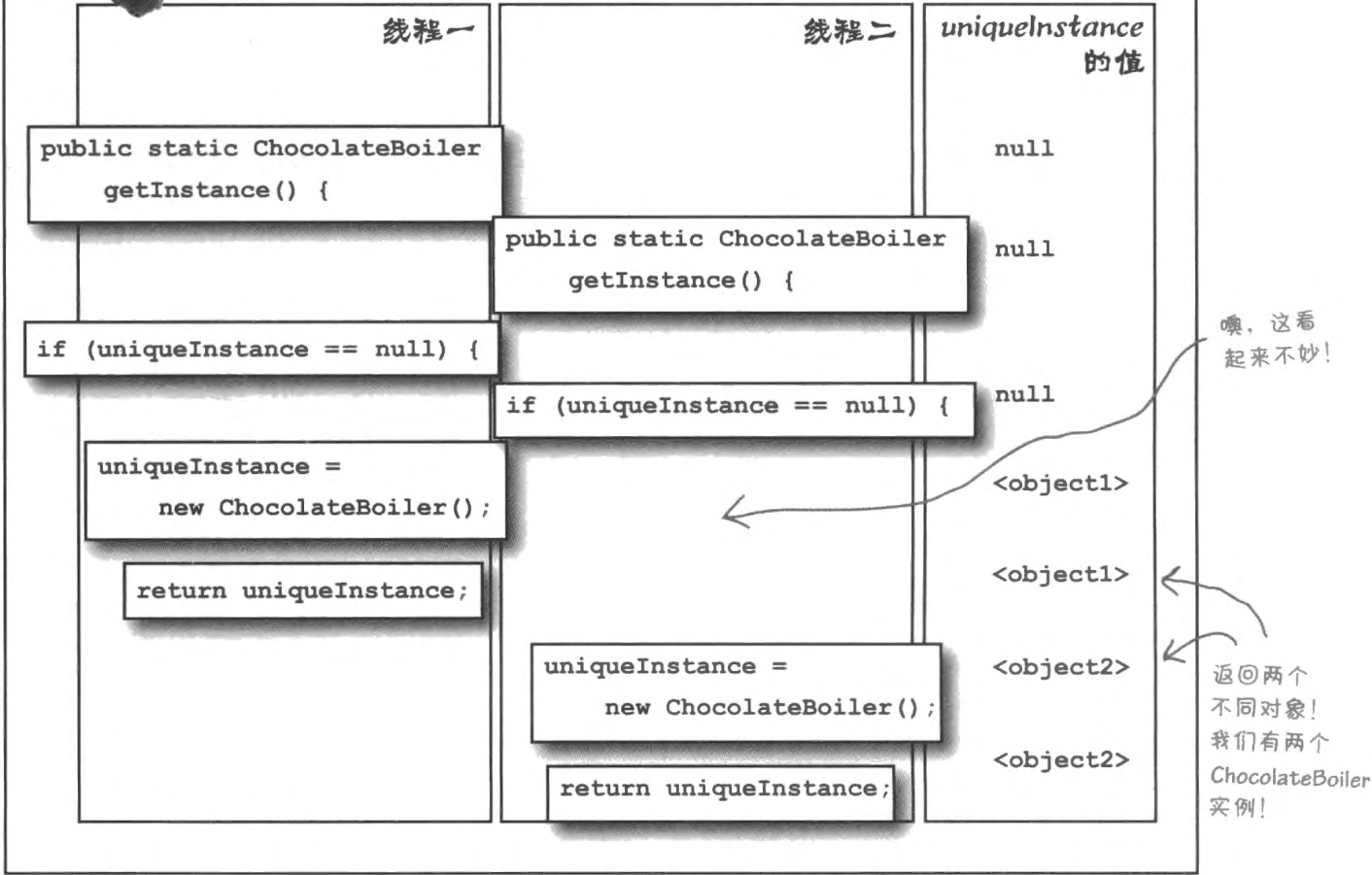
- 3. 生产锅炉的公司。
- 6. 不正确的实现导致这个溢出。
- 7. 单件模式有一个。
- 10. 为了完整打败new构造器，我们不得不声明构造器为_____。
- 12. 经典实现不处理这个。
- 13. 单件提供单一实例和_____（三个词）。
- 14. 在Java中创建单件的容易的方法。
- 15. 单件因没有公开的_____而尴尬。
- 16. 单件是一个管理_____的实例的类。

竖排

- 1. 往锅炉里添加巧克力。
- 2. 如果不使用Java 5或后续版本就会有缺陷的多线程方法（两个词）。
- 3. 它是“独一无二”的。
- 4. 多个_____（两个词）会导致出问题。
- 5. 如果你不需要操心延迟实例化，你可以创建你的实例。
- 8. 相对于全局变量的优点：_____创建。
- 9. 美国的巧克力之都。
- 11. 单件确保这些之中只有一个存在。



化身为JVM 答案





削尖你的铅笔

答案

请帮Choc-O-Holic改进ChocolateBoiler类，把它转成单件。

```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;

    private static ChocolateBoiler uniqueInstance;

    private ChocolateBoiler() {
        empty = true;
        boiled = false;
    }

    public static ChocolateBoiler getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new ChocolateBoiler();
        }
        return uniqueInstance;
    }

    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // 往锅炉里填充牛奶/巧克力混合物
        }
    }
    // 剩下的ChocolateBoiler代码……
}
```



削尖你的铅笔 答案

对于每个方案，描述其对修正巧克力锅炉代码问题的适用性：

同步getInstance()方法：

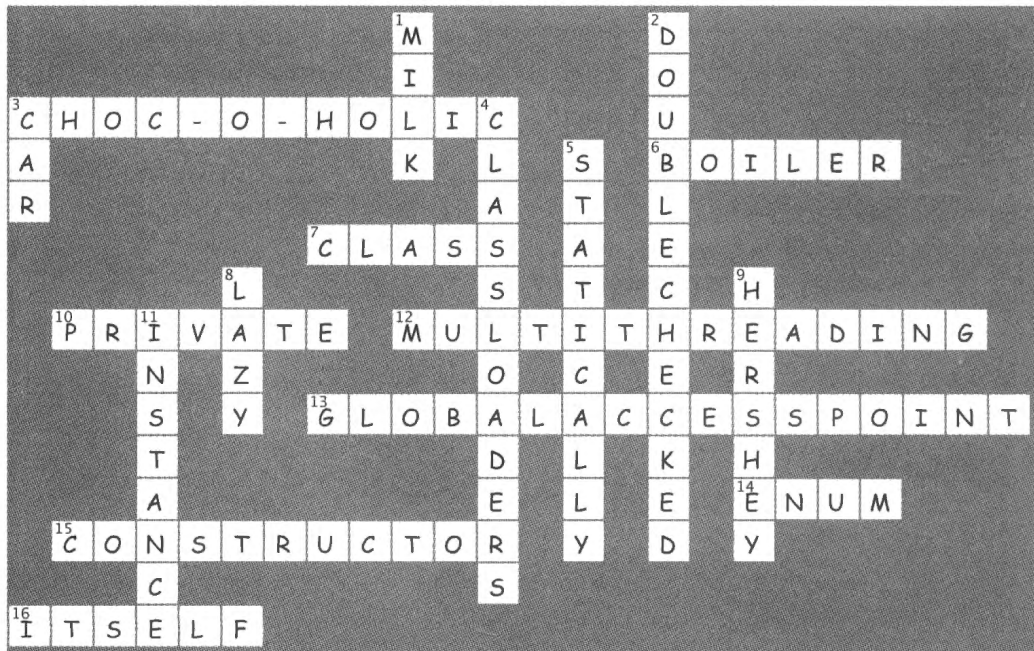
保证行得通的、直截了当的技巧。我们看起来不关心巧克力锅炉的性能问题，因此这是一个好的选择。

使用急切实例化：

我们总是打算在我们的代码中实例化一个巧克力锅炉，因此静态地初始化实例并无不可。这个方案和同步方法一样可行，虽然对于熟悉标准模式的开发人员来说，可能没那么容易理解。

双重检查加锁：

如果不考虑性能，双重检查加锁看起来是杀鸡用牛刀了。另外，我们不得不确保至少在Java 5以上运行。



设计模式
填字游戏
解答